실습: Solidity (1)

블록체인, 2026-2학기



김효승 (Hyoseung_kim@hallym.ac.kr)
정보과학대학 소프트웨어학부

- **학습목표**
  - 스마트 컨트랙트 기본 구조를 이해하고 설명한다
  - 변수 타입을 구분하고 적절히 사용한다
  - 값 타입과 참조 타입의 개념을 구분한다
  - 값 타입이 종류 별 특성을 이해하고 동작을 설명한다

- **솔리디티 (Solidity)**
  - 이더리움에서 계약을 프로그래밍하는 데 사용할 수 있는 도메인 특화 언어
    - 구문은 c에 가까움



  - 주요 특징
    - **정적 타입의 언어**로 컴파일 시 타입 검사 가능
    - 상속, 인터페이스, 라이브러리와 같은 **객체지향적 요소** 포함
  - 솔리디티 모노컬처
    - 거의 대부분이 솔리디티를 기반으로 스마트 계약을 구현
    - 상호운용성, 조합성, 개발 비용 vs 공통 취약점 내재, 고수준 언어의 비효율성

- **컨트랙트 예시**
  - 라이선스 식별자 / 지시문 / 컨트랙트 코드 영역으로 나눔

```
1  // SPDX 라이센스 식별자
2  // SPDX-License-Identifier: MIT
3
4  // 지시문
5  pragma solidity 0.6.8;
6  import "./MyLib.sol";
7
8  // 컨트랙트 코드
9  contract HelloWorld {
10     string private greet;
11
12     constructor(){
13         greet = "Hello World";
14     }
15
16     function getString() public view returns(string memory){
17         return greet;
18     }
19 }
```

- **라이선스 식별자**
  - 소스코드 이용 및 배포 조건을 명시
    - 미 표기 시 경고 (솔리디티 ver 0.6 이상)
  - 대표 라이선스
    - MIT: 저작권 고지를 하는 한 소스코드 복사·수정 ·배포 등 상업적 사용까지 허용
    - Apache-2.0: MIT와 비슷하며 특허 관련 조항이 명시되어 있음
    - GPL-3.0: 코드 수정·배포를 허용하지만 결과물도 동일한 GPL로 배포하고 코드 공개 필수
    - Unlicence: 저작권을 최대한 포기하고 누구나 자유롭게 사용하는 것을 허용
    - https://spdx.org/licenses/

```
1   // SPDX 라이센스 식별자
2   // SPDX-License-Identifier: MIT
```

- **지시문**
  - pragma: 지시한 조건의 버전의 컴파일러에서만 정상 동작한다는 의미

표현	설명
^0.8.3	0.8.x와 호환 (0.8.3 이상, 0.9.0 미만)
<0.8.3	0.8.3 미만과 호환
<=0.8.3	0.8.3 이하와 호환
>0.8.3	0.8.3 초과와 호환
>=0.8.3	0.8.3 이상과 호환
=0.8.3	정확히 0.8.3만 호환
0.8.3	=0.8.3과 동일
0.8.2 || 0.8.3	0.8.2 이거나 0.8.3
>=0.8.3 <0.9.0	0.8.3 이상, 0.9.0 미만과 호환

  - import: 다른 파일/라이브러리 코드 불러오기

```
4   // 지시문
5   pragma solidity 0.6.8;
6   import "./MyLib.sol";
```

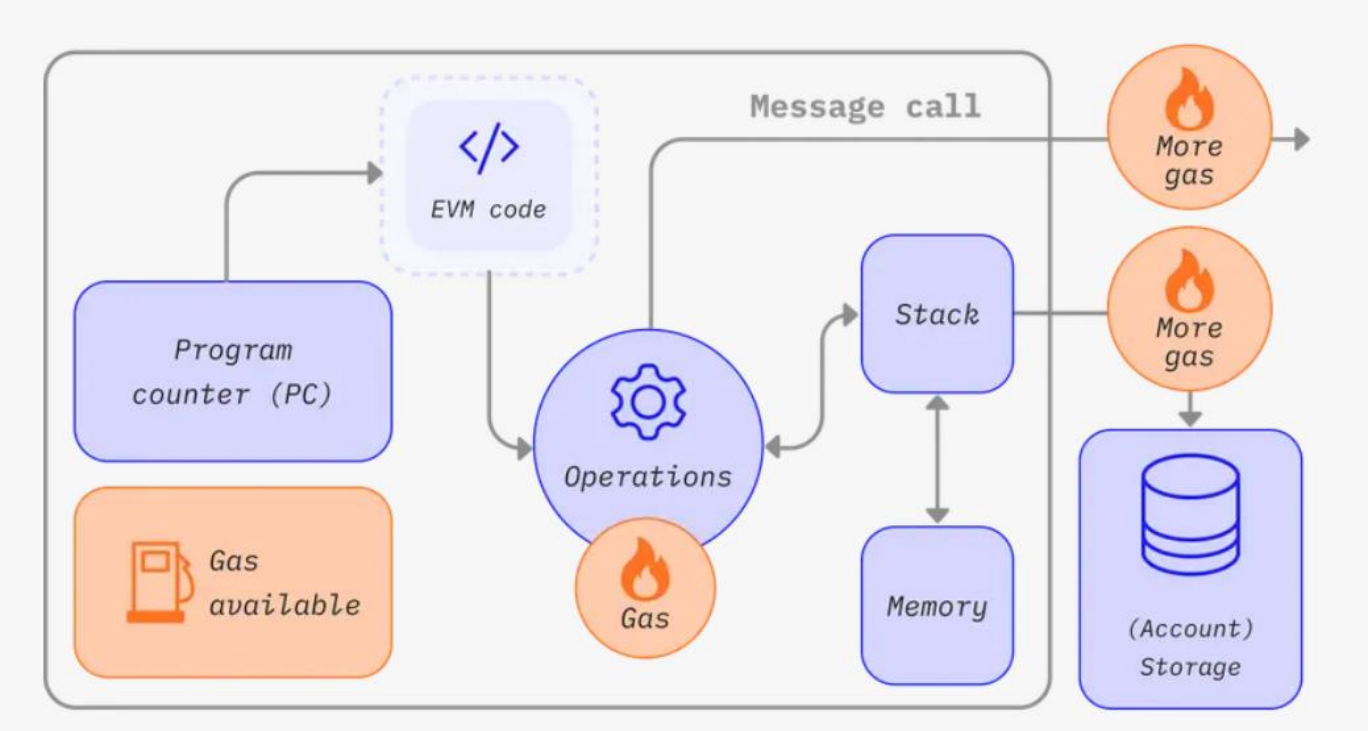
- **컨트랙트 코드**
  - 실제 동작을 정의하는 솔리디티 코드를 작성
  - 주요 포함 내용
    - 생성자: 컨트랙트가 배포될 때 한 번만 최초 실행되는 초기화 함수
    - 각종 타입의 변수: 전역변수, 상태변수, 지역변수
    - 기능 로직 구현을 위한 함수
    - 그 외 이벤트, 에러, 모디파이어 등 컨트랙트 동작을 구성하는 모든 코드

```
 9  contract HelloWorld {
10      string private greet;
11
12      constructor(){
13          greet = "Hello World";
14      }
15
16      function getString() public view returns(string memory){
17          return greet;
18      }
19  }
```

- **컨트랙트 코드**
  - 함수 표현 (preview)
    - 함수명, 접근제어자, 속성, 반환 타입을 지정

```
1   // SPDX-License-Identifier: MIT
2   pragma solidity ^0.8.0;
3
4   contract Function{
5
6       function returnMultiValues() public pure returns (uint, bool, uint){    infinite gas
7           return (1, true, 2);
8       }
9
10      function modifiedFirstFunction() public pure returns (uint x, bool b, uint y) {    infinite gas
11          x = 1; b = true; y = 2;
12      }
13
14      uint[] public arr;
15      function arrayInput(uint[] memory _arr) public{    infinite gas
16          arr = _arr;
17      }
18
19      function arrayOutput() public view returns(uint[] memory){    infinite gas
20          return arr;
21      }
22  }
```

- **EVM의 변수 저장 영역 (Data Location)**
  - 스토리지: 컨트랙트의 상태변수가 영구히 저장되는 공간
  - 메모리: 함수 실행 중에만 존재하는 임시 공간 (휘발성)
  - 호출 데이터: 외부에서 함수 호출할 때 입력 값이 저장되는 읽기 전용 임시 공간
  - 스택: EVM 내부의 계산용 임시 저장 공간. 1024 단계 이상 저장 시 예외 발생



- **전역 변수 (Global Variables)**
  - 블록체인 환경이 제공하는 특별 변수
    - 컨트랙트나 함수에 따로 선언하지 않아도 사용 가능
    - 읽기 전용이며 시스템 데이터를 다룸
    - 비슷한 개념으로 전역 함수도 존재
  - 예시
    - block.timestamp: 현재 블록 시간
    - block.number: 현재 블록 번호
    - msg.sender: 함수를 호출한 주소
    - msg.value: 함께 전송된 ETH양
    - tx.origin: 트랜잭션을 시작한 원래 주소
    - tx.gasprice: 트랜잭션 가스 가격

전역 변수 & 전역 함수에 대한 내용은 추후 더 자세히 다룰 예정

- **상태 변수 (State Variables)**
  - 블록체인에 영구적으로 저장되는 변수
    - 모든 함수에서 접근 가능하고 컨트랙트의 **상태를 나타냄**
    - 컨트랙트 내부, 함수 외부에 선언
    - 변수 값은 컨트랙트에 할당되는 고유 스토리지에 저장 (CA의 storageRoot값으로 식별)
    - 값을 **읽고 쓰는 데 모두 가스 소모**
    - 값이 변경되면 블록체인에 기록되어 다음 트랜잭션에서도 반영

- **지역 변수(Local Variables)**
  - 함수 내부에서 선언되고 실행이 끝나면 사라지는 변수
    - 스토리지에 저장되지 않고 스택이나 메모리에 존재
    - 읽고 쓰는 가스 비용이 드나 **상태 변수보다 상대적으로 매우 저렴**

- 예시

```
1   // SPDX-License-Identifier: MIT
2   pragma solidity >=0.8.0 <0.9.0;
3
4   contract HelloWorld {
5       // 상태 변수: 블록 체인에 기록
6       string public str = "Hello World";
7
8       function getInfo() public view returns(uint, uint, address){
9           // 지역 변수: 함수 안에서만 사용
10          uint num1 = 2;
11
12          // 전역 변수: 블록체인에 관한 정보 제공
13          uint num2 = block.timestamp;
14          address addr = msg.sender;
15
16          return (num1, num2, addr);
17      }
18  }
```

- **솔리디티 자료 타입**
  - 값 타입 (Value Type)
    - 변수 선언과 동시에 메모리를 생성해 **값(Value) 자체를 저장**
    - 복사 시 원본과 완전히 분리된 사본을 만듦
    - 복사본마다 독립적이며 **변경에 영향을 받지 않음**
    - 저장 위치를 자동으로 처리
      Ex) bool, uint, address 등
  - 참조 타입 (Reference Type)
    - 값을 변수에 직접 저장하지 않고 값이 저장된 위치의 **주소를 저장**하고 참조함
    - 복사 시 같은 저장 공간을 가리킴
    - 저장 위치(storage, memory, calldata)를 명시해야 함
    - 저장 위치에 따라 값이 변경되면 **모든 참조가 영향을 받음** (Ex. memory → memory)
      Ex) sting, array, struct, mapping 등

- **참조 타입 자료의 저장 영역에 따른 복사/참조**

소스	타겟	복사/참조 여부
memory	memory	참조
memory	storage	복사
storage	memory	복사
storage	storage	참조
calldata	memory	복사
calldata	storage	복사

- 같은 영역끼리 복사하면 같은 공간을 사용할 수 있으니 참조(얕은 복사)
  - 한쪽이 바뀌면 다른 쪽에도 영향

- **불리언 타입**
  - **Bool**: 참(True) 과 거짓(False) 중 하나를 나타내는 논리 타입
  - 사용 연산자
    - ! : 논리적 부정 NOT
    - && : 논리적 접속사 AND
    - || : 논리적 분리 OR
    - == : 같음
    - != : 같지 않음

```
contract Boolean{
    function getTrue() public pure returns (bool){
        bool a = true;
        bool b = false;
        return a || b;
    }

    function getFalse() public pure returns (bool) {
        bool a = false;
        bool b = true;
        return a && b;
    }
}
```

- **정수 타입**
  - **Int / uint** : 다양한 크기의 부호가 있는 정수 / 부호가 없는 정수

int	uint
int int8 : $-2^7 \sim 2^7 - 1$ int16: $-2^{15} \sim 2^{15} - 1$ int32: $-2^{31} \sim 2^{31} - 1$ int64: $-2^{63} \sim 2^{63} - 1$ int128: $-2^{127} \sim 2^{127} - 1$ int256(=int): $-2^{255} \sim 2^{255} - 1$	uint uint8 : $0 \sim 2^8 - 1$ uint16: $0 \sim 2^{16} - 1$ uint32: $0 \sim 2^{32} - 1$ uint64: $0 \sim 2^{64} - 1$ uint128: $0 \sim 2^{128} - 1$ uint256(=uint): $0 \sim 2^{256} - 1$

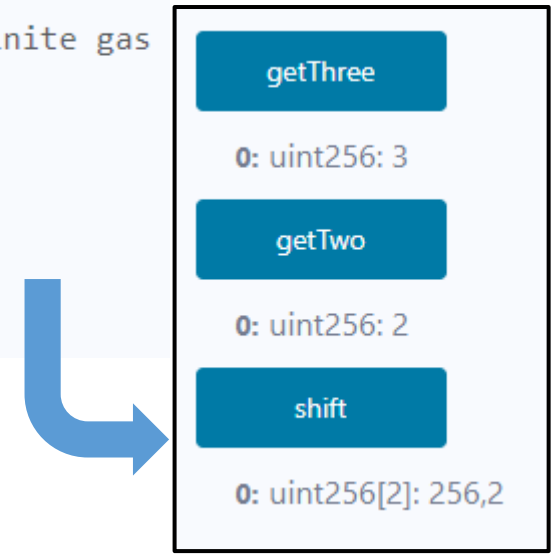
  - 사용 연산자
    - 비교연산자: <=, <, ==, !=, >=, >
    - 비트 연산자: &(AND), |(OR), ^(XOR), ~(NOT)
    - 시프트 연산자: <<, >>
    - 산술 연산자: +, -, *, /, %, **(지수 연산)

- 정수 타입

```
contract Integers{
    function getTwo() public pure returns(uint){    infinite gas
        uint a = 3;
        uint b = 2;
        return a / b * 2;
    }

    function getThree() public pure returns(uint){    310 gas
        return 3 / 2 * 2;
    }

    function shift() public pure returns(uint[2] memory){    infinite gas
        uint[2] memory a;
        a[0] = 16 << 4; // 16 * (2 ** 4)
        a[1] = 16 >> 3; // 16 / (2 ** 3)
        return a;
    }
}
```



- **주소 타입 (Address)**
  - 20 바이트(이더리움 주소 크기) 값을 저장
    - address 와 address payable 유형 존재
  - 정수 타입의 비교 연산자와 멤버 함수사용
    - balance: 계정 주소에 있는 이더를 wei로 리턴
    - transfer: 계정 주소에 있는 이더를 송금하고 실패 시 트랜잭션이 되돌려짐 //0.8.31 부터 deprecated
    - send: 계정 주소에 있는 이더를 송금하고 실패 시 false를 반환 //0.8.31 부터 deprecated
    - call: 다른 컨트랙트 함수 호출, 가스 설정 등 다양한 목적으로 사용 가능
    - delegatecall: 다른 컨트랙트의 코드만 빌려 쓰고 저장소는 현재 컨트랙트를 사용
  - 위의 함수 중 transfer, send, call(송금목적)은 **address payable 타입**에서만 가능

```
address contractAddress = address(this);          // 현재 CA 주소
address payable onwer = payable(msg.sender);      // 호출한 계정 또는 컨트랙트 주소
```

```
if(contractAddress.balance >= 1 ether){
    onwer.transfer(10 ** 18);
}
```

자료 타입 – 값 타입

```
contract Address{
    receive() external payable {}    undefined gas

    function getBalance(address addr) public view returns(uint){    3313 gas
        return addr.balance;
    }

    function transfer(address payable to, uint amount) public {    undefined gas
        to.transfer(amount);
    }

    function send(address payable to, uint amount) public {    undefined gas
        if(!to.send(amount)){
            revert("Send failed");
        }
    }

    function call(address payable to, uint amount) public {    undefined gas
        (bool success, ) = to.call{value: amount, gas: 100000}("");
        if(!success){
            revert("Call failed");
        }
    }

    function withDraw() public {    infinite gas
        address payable to = payable(msg.sender);
        to.transfer(address(0).balance);
    }
}
```

- **바이트 배열 타입**
  - bytes1, bytes2, … bytes32 까지 가능
  - 비교, 비트연산자, 시프트연산자, 인덱스 연산자 가능
  - 동적 크기의 바이트 배열은 참조 타입

```
contract Bytes{
    function fixedBytesArray() public pure returns (bytes2){    328 gas
        bytes2 b = "ba";
        return b;
    }

    function dynamicBytesArray() public pure returns (bytes memory){    infinite gas
        bytes memory a = 'baaaaaaaaaa'; //동적 크기의 배열 (참조타입)
        return a;
    }
}
```

  - b[0]='b', b[1]='a' 로 저장

- **열거 타입**
  - 유한한 상수 값으로 이뤄진 집합으로 사용자 정의 유형을 만드는데 사용

```
contract Enum {
    enum Colors {Red, Blue, Green}

    Colors color;
    Colors constant defaultColor = Colors.Green;

    function setColor() public {    24450 gas
        color = Colors.Blue;
    }

    function getColor() public view returns (Colors){    2560 gas
        return color;
    }

    function getDefaultcolor() public pure returns(uint){    336 gas
        return uint(defaultColor);
    }
}
```

  - setColor() 호출 하지 않으면 getColor()는 0을 리턴

- **함수 타입**
  - 함수 이름 그대로를 변수로 사용
  - 내부 타입(internal)과 외부 타입(external, public) 2가지 유형 존재
    - 내부 타입: 현재 함수 안에서 함수를 참조
    - 외부 타입: 주소를 통해 호출

```
contract Selector{
    function e() external pure returns(bytes4){    344 gas
        return this.e.selector;
        // this: 현재 컨트랙트를 가리키는 포인터
        // this.e: 외부에서 함수 e를 호출하는 외부 함수 타입
        // .selector: 함수의 4바이트 식별자
    }

    function f() public pure returns(uint){
        return g();
        // g(): 내부 함수 타입    344 gas
    }

    function g() internal pure returns(uint){
        return 100;
    }
}
```

- **참고 강의교안**
  - 처음 배우는 블록체인| 가사키 나가토, 시노하라 와타루 | 한빛미디어
  - Mastering Ethereum | Andreas M. Antonopoulos 외 4인 | O'Reilly Media

Thank You

Q&A